

Especificação de Requisitos de Software (ERS)

CtrlAccess

Controle de Versão

Versão	Data	Descrição das Alterações
1.0	21.02.2026	Emissão inicial do documento de requisitos e definição arquitetural.

1. Introdução

1.1 Objetivo do Documento

Este documento especifica os requisitos de software para o sistema **CtrlAccess**, definindo suas funcionalidades, arquitetura, restrições técnicas e métricas de qualidade. Este artefato serve como base fundamental para as fases de design, implementação e testes.

1.2 Escopo do Produto

O **CtrlAccess** é um sistema para rede local (LAN) projetado para o monitoramento e automação de equipamentos de controle de acesso (catracas, portas e terminais faciais). O sistema integra-se diretamente à API local de dispositivos da marca *Control iD*, gerenciando o ciclo de liberação, bloqueio e rotinas baseadas em regras de horário.

1.3 Aviso Legal e Dependências

O CtrlAccess é um software independente. Não possui vínculo, parceria ou endosso da empresa **Control iD**. A comunicação com o hardware é realizada estritamente via protocolo HTTP local (API nativa dos equipamentos), sob administração e responsabilidade da infraestrutura do usuário final.

2. Arquitetura do Sistema

2.1 Padrão Arquitetural

O sistema adotará uma arquitetura **Cliente-Servidor Desacoplada** organizada fisicamente em um **Monorepo**.

- **Backend (API RESTful):** Adotará arquitetura em camadas (SoC - *Separation of Concerns*), segregando explicitamente *Routes*, *Controllers*, *Services* e *Data Access*.
- **Frontend (SPA - Single Page Application):** Interface *client-side* com renderização e roteamento dinâmicos geridos no navegador, comunicando-se de forma estritamente assíncrona com a API.

2.2 Especificação Tecnológica

- **Plataforma Backend:** Node.js.
- **Framework Web (API):** Express.js (padronizado com ES Modules).
- **Camada de Persistência:** SQLite (via biblioteca better-sqlite3).
- **Processamento em Background:** node-cron (orquestração de tarefas agendadas).
- **Camada de Segurança:** JSON Web Tokens (JWT) para controle de sessão e

bcrypt para *hashing* de senhas.

- **Plataforma Frontend:** HTML5, CSS3 e JavaScript (Vanilla ES6+).
- **Comunicação de Rede:** Fetch API (Frontend) e axios / fetch (Backend para dispositivos físicos).

3. Requisitos Funcionais (RF)

Os requisitos funcionais descrevem as capacidades que o sistema deve prover.

Módulo: Gestão de Dispositivos

ID	Título	Descrição
RF-01	Manutenção de Dispositivos	O sistema deve prover interface (CRUD) para cadastrar, editar, excluir e listar dispositivos, armazenando: Nome, Grupo/Setor, IP, Porta, Usuário e Senha.
RF-02	Monitoramento de Status	O backend deve realizar verificações assíncronas de conectividade (ping/healthcheck) com os IPs cadastrados, refletindo o status atual (Online/Offline) na UI.
RF-03	Atuação Manual	O sistema deve expor endpoints e interface para disparo manual de comandos de rede: <i>Abrir Relé (Porta)</i> e <i>Reiniciar Dispositivo</i> .
RF-04	Alteração de Estado de Operação	O sistema deve permitir a mudança de estado lógico dos terminais entre: <i>Normal</i> (validação requerida), <i>Liberado/Emergency</i> (passe livre) e <i>Trancado/Lockdown</i> (acesso negado globalmente).

Módulo: Automação e Regras (Schedules)

ID	Título	Descrição
RF-05	Manutenção de Regras	O sistema deve permitir a gestão de rotinas, registrando: Título, Grupo Alvo (Setor), Dias da Semana, Horário

		Início/Fim e Ação a Executar.
RF-06	Motor de Automação (Worker)	O sistema deve instanciar um <i>worker job</i> no servidor para avaliar continuamente a tabela de regras e enviar comandos aos dispositivos no momento exato do <i>timestamp</i> configurado.

Módulo: Segurança e Acesso

ID	Título	Descrição
RF-07	Autenticação e Autorização	O sistema deve exigir autenticação via login. Todas as rotas de mutação de estado e acesso ao <i>dashboard</i> devem validar a presença e integridade do token JWT.

4. Requisitos Não Funcionais (RNF)

Os requisitos não funcionais definem as restrições e os atributos de qualidade do sistema.

ID	Categoria	Descrição
RNF-01	Ambiente	A aplicação deve ser hospedada e executada integralmente em Rede Local (Intranet/LAN) para garantir alcance aos IPs privados do hardware.
RNF-02	Segurança	As credenciais (usuário e senha) dos dispositivos físicos armazenadas no banco de dados não devem ser expostas ou trafegadas para o frontend (payloads HTTP).
RNF-03	Desempenho	O processo de avaliação do cron (RF-06) deve operar de forma não-bloqueante ao Event Loop do Node.js, com periodicidade de 60 segundos (****).
RNF-04	Usabilidade	A interface do usuário deve aplicar o padrão SPA. Submissões de formulários e

		acionamentos de relés não podem gerar recarregamento da página (<i>full page reload</i>).
RNF-05	Manutenibilidade	A base de código backend deve adotar injeção ou passagem de dependências nas rotas, isolando estritamente as regras de negócio (/services) da definição de <i>endpoints</i> (/routes).

5. Estrutura do Repositório (Codebase)

A padronização dos diretórios reflete as restrições do Padrão Arquitetural e isolamento de escopo (SoC).

```

/
├── data/
│   └── database.sqlite      # Instância física do banco de dados relacional
├── src/
│   ├── backend/
│   │   ├── config/        # Definições de ambiente (ENV) e constantes
│   │   ├── routes/        # Declaração de rotas e acoplamento de middlewares
│   │   ├── controllers/    # Validação de I/O (Requests/Responses HTTP)
│   │   ├── services/       # Regras de negócio, Integração de Hardware e Cron
│   │   ├── middlewares/    # Interceptadores
│   │   ├── app.js          # Bootstrap da aplicação Express
│   │   └── server.js       # Entrypoint do servidor Node.js
│   └── frontend/
│       ├── index.html      # Entrypoint SPA (DOM Container)
│       ├── css/            # Folhas de estilo (Global e Componentes)
│       └── js/
│           ├── app.js       # Orquestrador de estado e roteamento Client-Side
│           ├── views/       # Controladores de visualização (Telas)
│           ├── components/   # Componentes de UI reutilizáveis e isolados
│           └── services/     # Clientes de API (Fetch requests)

```